

Lab_11

Kenya Sanchez

Sections 1-4 were discussed in Class

Section 5: The EBI AlphaFold database

Querying the AlphaFold database

Familiarize ourselves with AlphaFold DataBase

Start by looking for HIV-Protease structure in the AlphaFold Database by searching an HIV-PR sequence and examining its top hits.

My Top Hit: “Gag-Pol polyprotein” from *Human immunodeficiency virus type 1 group M subtype B* with id *P04585*

Section 6: Generating your own structure predictions

Getting Started with ColabFold

Performed the HIV-Pr-Monomer input for my first Colab Notebook. The monomer version of this protein is unstable in reality but its dimerized conformation is the functional unit. We used the monomer version for the first run because it is easier to superimpose this single chain model on molstar.

Zip file automatically downloaded after this procedure.

Section 7: Interpreting Results

Visualization of the models and their estimated reliability

Using Mol (molstar)

Below is the image generated for Figure 18

HIV-Pr-Monomer;

```
# Figure 18: HivPr- Monomer
```

```
knitr::include_graphics("Fig.18_monomer_HIVPR_94B5B.png")
```



Section 8: Custom analysis of resulting models

Creating a vector for the extended zip file which contains our hivpr-dimer information. Read results for HIV-Pr-Dimer in R using the Bio3D package.

```
# Results dir name & Vector
results_dir <- "hivprdimer_23119"
```

Create Vector for all pdb files. There should be a total of 5 pdbs:

```
# File names for all PDB models
pdb_files <- list.files(path=results_dir,
                        pattern="*.pdb",
                        full.names = TRUE)

# Print our PDB file names
basename(pdb_files)
```

```
[1] "hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb"
[2] "hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000.pdb"
[3] "hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000.pdb"
[4] "hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000.pdb"
[5] "hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb"
```

Use `pdbaln()` function to align the amino acid sequences so R knows which residue in Protein A corresponds to which residue in Protein B.

```
library(bio3d)

# Read all data from Models
# and superpose/fit coords
pdbs <- pdbaln(pdb_files, fit=TRUE, exefile="msa")
```

Reading PDB files:

```
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000
.....
```

Extracting sequences

```
pdb/seq: 1   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v
pdb/seq: 2   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v
pdb/seq: 3   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v
pdb/seq: 4   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v
pdb/seq: 5   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v
```

Viewing Model Sequences

All sequences should be the same

```
# pdb Sequences
```

```
pdbs
```

```

1                                     .               .               .               .               50
[Truncated_Name:1]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGI
[Truncated_Name:2]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGI
[Truncated_Name:3]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGI
[Truncated_Name:4]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGI
[Truncated_Name:5]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGI
*****
1                                     .               .               .               .               50

51                                   .               .               .               .               100
[Truncated_Name:1]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:2]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:3]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:4]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:5]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
*****
51                                   .               .               .               .               100

101                                  .               .               .               .               150
[Truncated_Name:1]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGIG
[Truncated_Name:2]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGIG
[Truncated_Name:3]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGIG
[Truncated_Name:4]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGIG
[Truncated_Name:5]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWPKPMIGGIG
*****
101                                  .               .               .               .               150
```

```

                                151          .          .          .          .          198
[Truncated_Name:1]hivprdimer  GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:2]hivprdimer  GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:3]hivprdimer  GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:4]hivprdimer  GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:5]hivprdimer  GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
                                *****
                                151          .          .          .          .          198

```

Call:

```
pdbaln(files = pdb_files, fit = TRUE, exefile = "msa")
```

Class:

```
pdb, fasta
```

Alignment dimensions:

```
5 sequence rows; 198 position columns (198 non-gap, 0 gap)
```

```
+ attr: xyz, resno, b, chain, id, ali, resid, sse, call
```

Use the `rmsd()` function to calculate the RMSD between all pairs models.

```
# Calculate Root Mean Square Deviation (RMSD) for aligned protein structures
rd <- rmsd(pdb, fit=T)
```

Warning in `rmsd(pdb, fit = T)`: No indices provided, using the 198 non NA positions

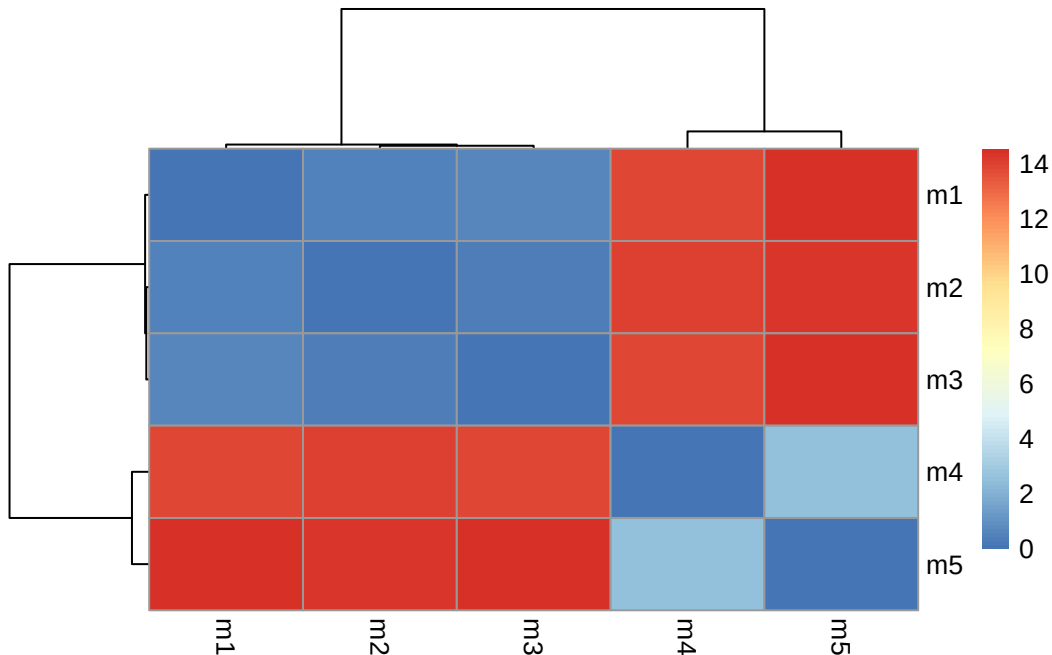
```
# Use range function for max and min RMSD values
range(rd)
```

```
[1] 0.000 14.526
```

Heatmap of RMSD matrix values

```
library(pheatmap)

colnames(rd) <- paste0("m",1:5)
rownames(rd) <- paste0("m",1:5)
pheatmap(rd)
```



Scale represents structural distance between models 1-5 shown as m1-m5.

Note: models 1 and 2 are more similar to each other than they are to any other model. Models 4 and 5 are quite similar to each other and in turn more similar to model 3 than to models 1 and 2.

Plot pLDDT values

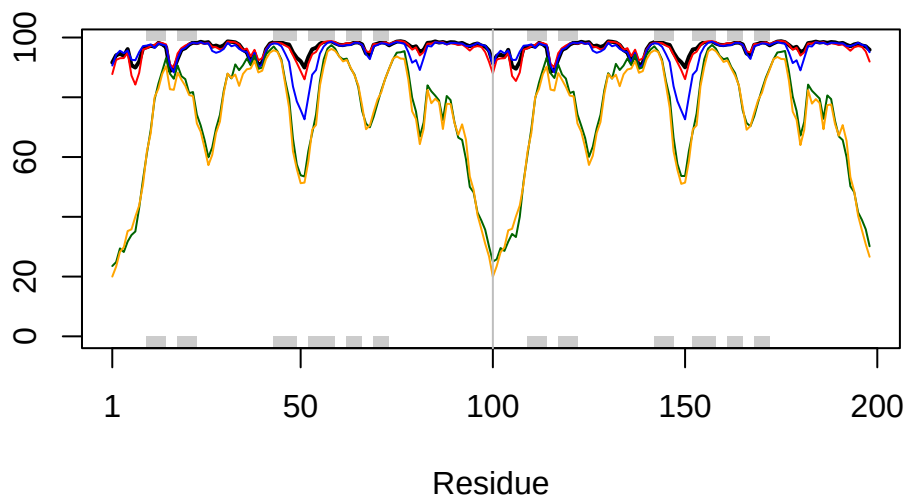
```
# Read a reference PDB structure
pdb <- read.pdb("1hsg")
```

Note: Accessing on-line PDB file

Flexibility Profile Plot:

Used to compare the stability and movement of specific regions across your five different protein models

```
plotb3(pdb$b[1,], typ="l", lwd=2, sse=pdb)
points(pdb$b[2,], typ="l", col="red")
points(pdb$b[3,], typ="l", col="blue")
points(pdb$b[4,], typ="l", col="darkgreen")
points(pdb$b[5,], typ="l", col="orange")
abline(v=100, col="gray")
```



Improve superposition model using core.find

```
# Find most consistent "rigid core"
core <- core.find(pdb)
```

```
core size 197 of 198 vol = 5311.905
core size 196 of 198 vol = 4622.32
core size 195 of 198 vol = 1803.028
core size 194 of 198 vol = 1106.755
core size 193 of 198 vol = 1033.427
core size 192 of 198 vol = 985.198
core size 191 of 198 vol = 939.616
core size 190 of 198 vol = 896.766
core size 189 of 198 vol = 856.252
core size 188 of 198 vol = 822.829
core size 187 of 198 vol = 791.898
core size 186 of 198 vol = 762.687
core size 185 of 198 vol = 739.51
core size 184 of 198 vol = 712.268
core size 183 of 198 vol = 688.778
core size 182 of 198 vol = 669.224
core size 181 of 198 vol = 633.377
core size 180 of 198 vol = 614.667
```

core size 179 of 198	vol = 598.623
core size 178 of 198	vol = 582.74
core size 177 of 198	vol = 567.927
core size 176 of 198	vol = 553.536
core size 175 of 198	vol = 535.287
core size 174 of 198	vol = 522.628
core size 173 of 198	vol = 493.693
core size 172 of 198	vol = 479.665
core size 171 of 198	vol = 462.315
core size 170 of 198	vol = 448.76
core size 169 of 198	vol = 433.241
core size 168 of 198	vol = 420.053
core size 167 of 198	vol = 410.378
core size 166 of 198	vol = 398.085
core size 165 of 198	vol = 387.24
core size 164 of 198	vol = 373.948
core size 163 of 198	vol = 362.751
core size 162 of 198	vol = 348.611
core size 161 of 198	vol = 336.713
core size 160 of 198	vol = 324.778
core size 159 of 198	vol = 314.115
core size 158 of 198	vol = 302.268
core size 157 of 198	vol = 291.431
core size 156 of 198	vol = 280.553
core size 155 of 198	vol = 271.537
core size 154 of 198	vol = 262.651
core size 153 of 198	vol = 253.338
core size 152 of 198	vol = 244.266
core size 151 of 198	vol = 235.874
core size 150 of 198	vol = 228.071
core size 149 of 198	vol = 216.409
core size 148 of 198	vol = 203.865
core size 147 of 198	vol = 193.565
core size 146 of 198	vol = 185.893
core size 145 of 198	vol = 178.463
core size 144 of 198	vol = 170.348
core size 143 of 198	vol = 163.308
core size 142 of 198	vol = 153.436
core size 141 of 198	vol = 143.699
core size 140 of 198	vol = 137.681
core size 139 of 198	vol = 132.232
core size 138 of 198	vol = 124.684
core size 137 of 198	vol = 117.401

core size 136 of 198	vol = 109.876
core size 135 of 198	vol = 104.889
core size 134 of 198	vol = 98.885
core size 133 of 198	vol = 95.016
core size 132 of 198	vol = 91.683
core size 131 of 198	vol = 87.816
core size 130 of 198	vol = 84.046
core size 129 of 198	vol = 79.701
core size 128 of 198	vol = 76.896
core size 127 of 198	vol = 73.948
core size 126 of 198	vol = 70.293
core size 125 of 198	vol = 66.763
core size 124 of 198	vol = 64.12
core size 123 of 198	vol = 60.644
core size 122 of 198	vol = 57.631
core size 121 of 198	vol = 53.838
core size 120 of 198	vol = 50.347
core size 119 of 198	vol = 46.207
core size 118 of 198	vol = 43.507
core size 117 of 198	vol = 39.709
core size 116 of 198	vol = 36.796
core size 115 of 198	vol = 34.65
core size 114 of 198	vol = 31.443
core size 113 of 198	vol = 28.79
core size 112 of 198	vol = 26.627
core size 111 of 198	vol = 24.28
core size 110 of 198	vol = 22.088
core size 109 of 198	vol = 20.127
core size 108 of 198	vol = 18.254
core size 107 of 198	vol = 16.785
core size 106 of 198	vol = 15.453
core size 105 of 198	vol = 14.137
core size 104 of 198	vol = 12.84
core size 103 of 198	vol = 11.04
core size 102 of 198	vol = 10.012
core size 101 of 198	vol = 9.055
core size 100 of 198	vol = 8.017
core size 99 of 198	vol = 7.503
core size 98 of 198	vol = 6.268
core size 97 of 198	vol = 5.279
core size 96 of 198	vol = 4.152
core size 95 of 198	vol = 3.452
core size 94 of 198	vol = 2.854

```

core size 93 of 198  vol = 2.266
core size 92 of 198  vol = 1.944
core size 91 of 198  vol = 1.623
core size 90 of 198  vol = 1.382
core size 89 of 198  vol = 1.063
core size 88 of 198  vol = 0.873
core size 87 of 198  vol = 0.697
core size 86 of 198  vol = 0.552
core size 85 of 198  vol = 0.442
FINISHED: Min vol ( 0.5 ) reached

```

```

# Basis for more suitable superpositions
core.inds <- print(core, vol=0.5)

```

```

# 86 positions (cumulative volume <= 0.5 Angstrom^3)
  start end length
1      9  50     42
2     52  95     44

```

Generate physical PDB files

```

# These pdb structures accessible as "corefit_structures"
xyz <- pdbfit(pdb, core.inds, outpath="corefit_structures")

```

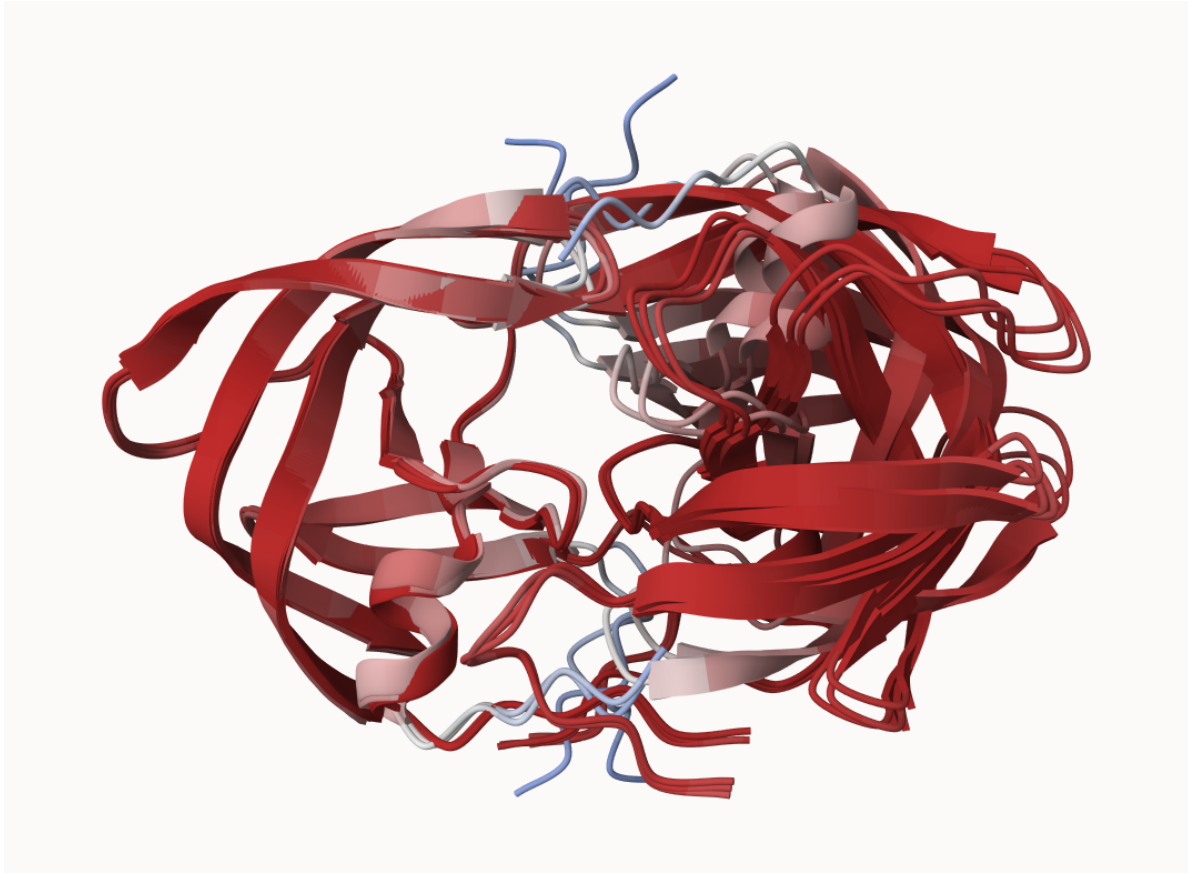
Generating **Figure 19**

This image only superimposes on one of the two chains because molstar does not allow for a perfect superimposition of both chains as the second structure is much more variable.

```

# Figure 19; Superimposed HIV-Pr-Dimer
knitr::include_graphics("Fig.19_HIVPRDIMER_23119.png")

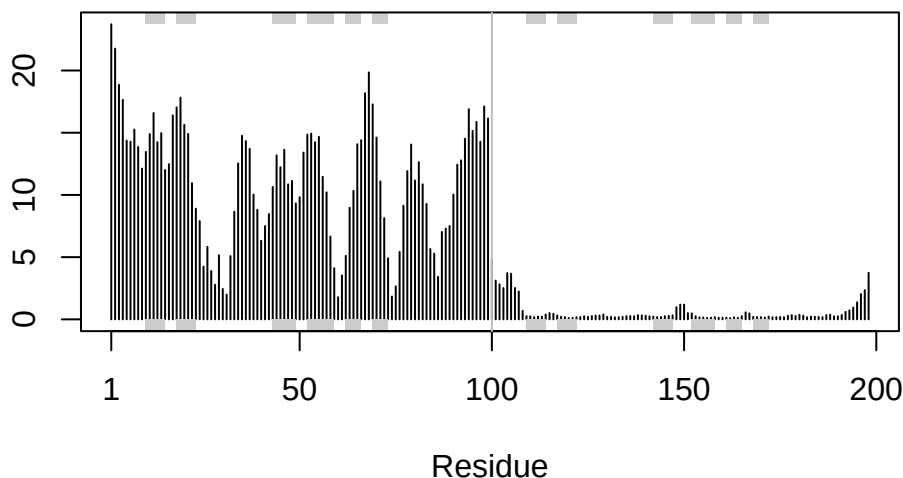
```



Examine the RMSF between positions of the structure

```
rf <- rmsf(xyz)

plotb3(rf, sse=pdb)
abline(v=100, col="gray", ylab="RMSF")
```



Note: Here we see that the first chain is largely very similar across the different models. However, the second chain is much more variable - we saw this in Molstar previously

Predicted Alignment Error for domains

Read PAE files

```
library(jsonlite)

# Listing of all PAE JSON files
pae_files <- list.files(path=results_dir,
                        pattern=".*model.*\\.json",
                        full.names = TRUE)
```

Note: AlphaFold produces a useful inter-domain prediction for model 1 (and 2) but not for model 5 (or indeed models 3, 4, and 5)

Read First and Fifth file

```
# Call out first and 5th files as a vector and read attributes
pae1 <- read_json(pae_files[1],simplifyVector = TRUE)
pae5 <- read_json(pae_files[5],simplifyVector = TRUE)
```

```
attributes(pae1)
```

```
$names
```

```
[1] "plddt" "max_pae" "pae" "ptm" "iptm"
```

```
# Per-residue pLDDT scores
```

```
# same as B-factor of PDB..
```

```
head(pae1$plddt)
```

```
[1] 91.62 94.06 94.56 93.88 96.12 90.69
```

Maximum PAE values are useful for ranking models

```
# Max PAE value for model 1
```

```
pae1$max_pae
```

```
[1] 12.33594
```

```
# Max PAE value for model 5
```

```
pae5$max_pae
```

```
[1] 29.45312
```

Note: Model 5 is much worse than model 1. The lower the PAE score the better

Question: How about the other models, what are there max PAE scores?

Answer: Out of all five models, model 4 is the one with the highest PAE score being 29.57812. Their ranking from lowest to highest (best to worst) are as follows: M1 (max_PAE = 12.33594), M3 (max_PAE = 14.39062), M2 (max_PAE = 17.48438), M5 (max_PAE = 29.45312), and M4 (max_PAE = 29.57812)

```
# Read Models 2-4
```

```
pae2 <- read_json(pae_files[2],simplifyVector = TRUE)
```

```
pae3 <- read_json(pae_files[3],simplifyVector = TRUE)
```

```
pae4 <- read_json(pae_files[4],simplifyVector = TRUE)
```

```
# Max PAE values for all models
```

```
pae1$max_pae
```

```
[1] 12.33594
```

```
pae2$max_pae
```

```
[1] 17.48438
```

```
pae3$max_pae
```

```
[1] 14.39062
```

```
pae4$max_pae
```

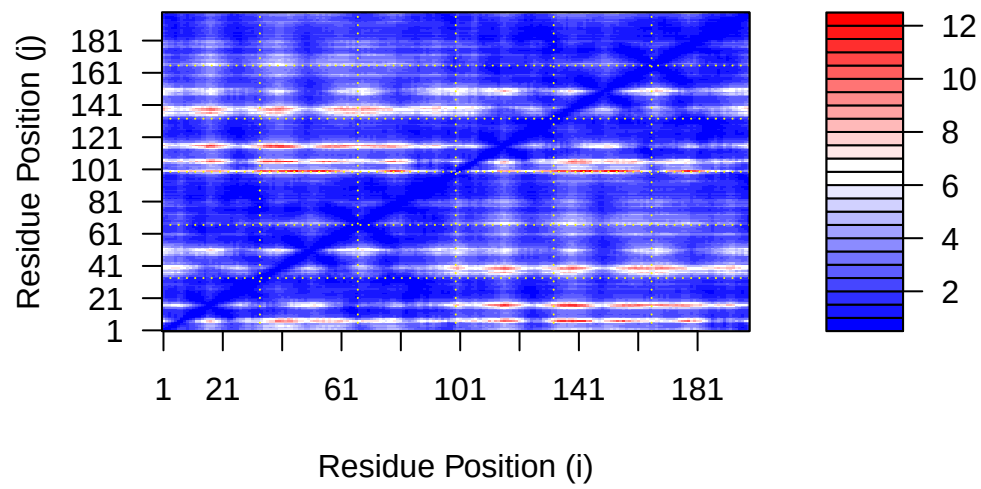
```
[1] 29.57812
```

```
pae5$max_pae
```

```
[1] 29.45312
```

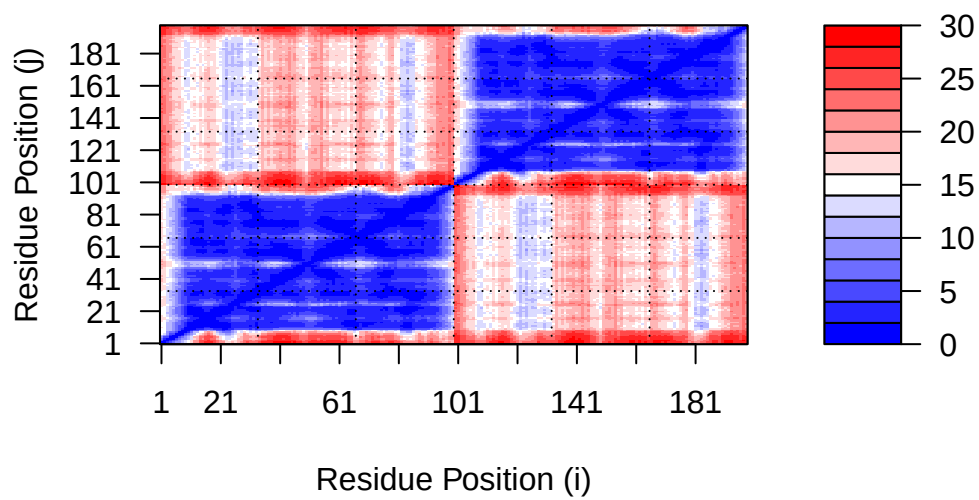
Plot the N by N (where N is the number of residues) PAE scores with functions from the Bio3D package:

```
# Generate PAE heatmap Model 1
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)")
```

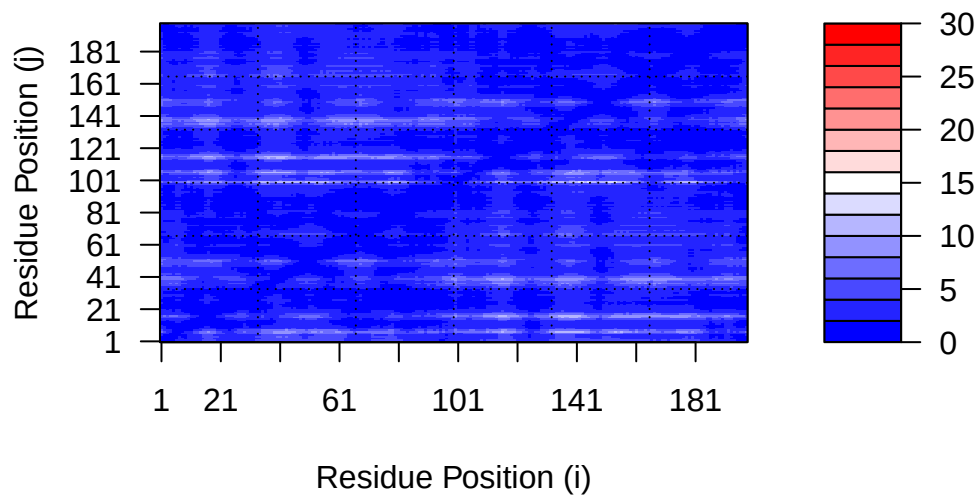


```
# Generate PAE heatmap Model 5

plot.dmat(pae5$paes,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



```
# Generate PAE heatmap Model 1 with same z range as Model 5
plot.dmat(pae1$pae,
  xlab="Residue Position (i)",
  ylab="Residue Position (j)",
  grid.col = "black",
  zlim=c(0,30))
```

Residue conservation from alignment file

File Finding Mission

```
# Locate specific alignment files generated by AlphaFold
aln_file <- list.files(path=results_dir,
                      pattern=".a3m$",
                      full.names = TRUE)

aln_file
```

```
[1] "hivprdimer_23119/hivprdimer_23119.a3m"
```

Move Localized File into R

```
# Read alignment file
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
[1] " ** Duplicated sequence id's: 101 **"
[2] " ** Duplicated sequence id's: 101 **"
```

How many sequences are in this alignment

There are **5397 sequences** being aligned

```
dim(aln$ali)
```

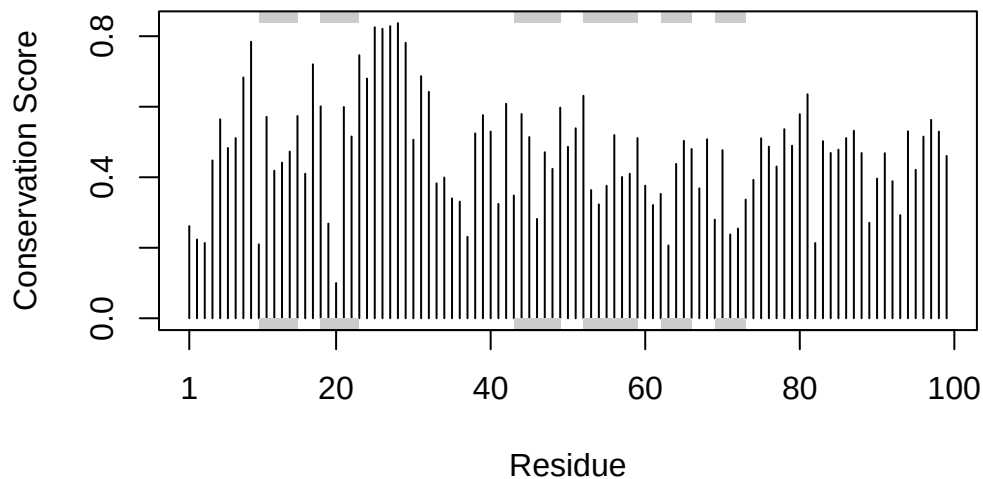
```
[1] 5397 132
```

Score residue conservation

```
# Score residue conservation in alignment using conserv() function  
sim <- conserv(aln)
```

Plot Conservation Scores

```
# Plot of Residue vs. Conservation Score  
plotb3(sim[1:99], sse=trim.pdb(pdb, chain="A"),  
        ylab="Conservation Score")
```



Cut off for 90% conservation to determine residues with high cutoff values

```
# Cut off value of 0.9  
con <- consensus(aln, cutoff = 0.9)  
con$seq
```

```

[1] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[19] "-" "-" "-" "-" "-" "-" "D" "T" "G" "A" "-" "-" "-" "-" "-" "-" "-" "-"
[37] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[55] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[73] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[91] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[109] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[127] "-" "-" "-" "-" "-" "-"

```

Map this conservation score to the Occupancy column of a PDB file for viewing in molstar.

```

m1.pdb <- read.pdb(pdb_files[1])
occ <- vec2resno(c(sim[1:99], sim[1:99]), m1.pdb$atom$resno)
write.pdb(m1.pdb, o=occ, file="m1_conserv.pdb")

```

Access this in molstar to generate fig.20

Image / Screenshot for Figure 20 shown Below

```

# Figure 20; Colored by Sequence Conservation
# DTGA motif of one chain is highlighted in green
knitr::include_graphics("Fig.20_M1_CONSERV.PDB.png")

```

